

opencode-memory

Plugin di memoria a lungo termine per opencode (ciclo WRITE → DREAM → SURFACE) —
Manuale tecnico e di funzionamento · versione 1.1 · 6 agosto 2026 · plugin: ~/ .config/
opencode/plugins/memory.ts · stato: **release candidate (plugin standalone)**

1. Panoramica

Il plugin dà a opencode una memoria persistente tra sessioni diverse, senza dipendere dal gestore di memorie del modello. Tre fasi, in un ciclo naturale:

- **WRITE** — cinque tool (memory_*) permettono di salvare, aggiornare, cancellare e leggere esplicitamente i fatti. **tool**
- **DREAM** — a fine conversazione il plugin rilegge il trascritto con una sessione LLM figlia (headless, invisibile nel picker) ed estrae fatti, aggiornamenti, cancellazioni e un riassunto. **headless**
- **SURFACE** — a ogni richiesta LLM il plugin inietta un blocco <memory> nel system prompt con i fatti rilevanti per il topic corrente. **iniezione**

Lo storage è **locale**: JSON atomici + lock file nella directory del plugin. Nessun server aggiuntivo, nessun database esterno, nessun servizio cloud del plugin.

Privacy — storage locale ≠ inference locale: lo storage è sempre locale, ma l'elaborazione LLM (DREAM, dedup, summary) usa il provider configurato in opencode: se il provider è remoto (es. API cloud), il testo delle conversazioni viene inviato a quel provider. Chi esegue il plugin su dati sensibili deve scegliere un provider locale (es. Ollama) oppure disattivare DREAM. Il plugin non aggiunge di suo alcun trasferimento: tutto passa dai canali standard di opencode verso il modello che l'utente ha già scelto.

2. Architettura

2.1 Componenti interni (memory.ts)

Componente	Ruolo
withLock / LOCK_FILE	

	Sezione critica cross-processo (lock via <code>open("wx")</code> con <code>retry</code>): tutte le scritture su <code>store/state</code> sono serializzate.
<code>getStore / readStore / writeStore</code>	Lettura sempre fresca dal disco (niente cache: le scritture della consolidazione non la aggiornerebbero) e scrittura atomica (<code>writeFile tmp + rename</code>).
<code>score / prune</code>	Ranking: $(weight + 0.5 \text{ se explicit}) \times \exp(-\text{giorni}/45)$; potatura delle entry vecchie o a peso basso.
<code>findSimilar / similarity / norm / topicClash</code>	Dedup lessicale: normalizzazione Unicode, sovrapposizione token, blocco a livello di topic.
<code>expandTopicKeywords</code>	Sinonimi bilingue (EN/IT) per il retrieval tematico (vedi 3.3).
<code>scheduleConsolidation / queue / pump / consolidate</code>	Dream headless in coda seriale (una consolidazione alla volta).
<code>dedupCheck</code>	Secondo pass LLM di deduplicazione semantica dei candidati (vedi 3.2).
<code>sweep / gcConsolidationChildren</code>	Scansione periodica: sessioni mai consolidate + garbage collection delle child orfane (vedi 3.2).
<code>buildMemoryBlock / sessionTopicKeywords</code>	SURFACE: selezione dei fatti per topic + nucleo permanente, iniezione in <code>output.system</code> .

2.2 Strutture su disco

```
DATA_DIR/
  store.json      # fatti + summary + versione
  state.json      # sessionID → lastTs consolidato
  summary.md      # proiezione leggibile del summary
  .lock           # lockfile (temporaneo)
```

3. Il ciclo di vita della memoria

3.1 WRITE — scrittura esplicita tool

Cinque tool sono registrati col prefisso `memory_` e sono quindi usabili dall'assistente in qualsiasi conversazione:

Tool	Args	Comportamento
<code>memory_read</code>		Restituisce summary + fatti filtrati/ordinati per rilevanza.

(titolo memory-consolidation, età > soglia E nessuna attività, non referenziate da un inProgress valido)

Gerarchia delle fonti e dedup semantico

Il cuore della qualità della memoria. Tre meccanismi sovrapposti:

Livello	Meccanismo	Regole
1. Prompt	Regole esplicite al modello di consolidazione	Mai aggiungere un fatto semanticamente equivalente a un entry explicit (stesso significato, qualsiasi lingua, parafrasi: "like green" == "preferisce il verde"); mai cancellare/aggiornare entry explicit; un nuovo fatto che duplica un dreamed deve mettere l'id vecchio in delete; un dreamed che contraddice un explicit va in delete.
2. Secondo pass (LLM)	dedupCheck: mini-chiamata dedicata nella stessa child headless	Se la consolidazione produce candidati, un prompt specializzato ("deduplication checker") elenca le entries esistenti e chiede "quali candidati sono duplicati?" → i duplicati non vengono aggiunti. Necessario perché i modelli piccoli ignorano le regole lunghe del pass 1.
3. Codice	findSimilar(0.5) + topicClash	Difesa in profondità: applyConsolidation non aggiunge mai un fatto simile a un explicit; non applica update/delete a entry explicit; blocco per "stessa categoria + parola di contenuto condivisa".

I conflitti NON entrano nel summary: il summary viene iniettato in ogni sessione futura, quindi menzionarvi un fatto in conflitto con una memoria esplicita lo farebbe rientrare dalla porta secondaria. Regola: i conflitti con explicit sono **ignorati completamente** — né come entry, né nel summary. (Prima versione: "segnalare nel summary" — rimossa.)

Verificato end-to-end: conversazione "my favorite color is green and I code in Rust" contro store con explicit "green" e dreamed "scrive codice in Rust" (IT) → nessun duplicato aggiunto, e il duplicato IT preesistente viene **ripulito** (delete del dreamed).

Perché la consolidazione è headless (invisibile)

Scoperta chiave: il selettore sessioni della TUI di opencode elenca le sessioni con roots: true — cioè **solo le sessioni radice** (packages/tui/src/component/dialog-session-list.tsx). Una sessione

creata con parentID è una figlia: non appare nel picker dell'utente, pur essendo un'istanza LLM perfettamente funzionante e tracciabile dal server.

Verifica empirica (HTTP): una child viva **non** compare in GET /session? roots=true, ma è presente nel list completo. Dopo il DELETE /session/{id} sparisce ovunque. Il consolidamento quindi non "sporca" la cronologia dell'utente.

Garbage collection delle child orfane

Se il processo muore dopo session.create ma prima del finally, la child resta (invisibile, ma presente) e l'inProgress resta non committato: al riavvio lo sweep lo vede scaduto e riconsolido la parent (nessuna perdita). Il garbage collector gcConsolidationChildren (eseguito a ogni sweep) elimina le sessioni con titolo memory-consolidation che soddisfano **entrambe** le condizioni: età > gcChildAgeMs (default 10 min) E nessuna attività recente (time.updated oltre soglia) — una consolidazione lenta ma viva non viene mai uccisa. Sono inoltre esenti le child referenziate da un inProgress non scaduto (anche di un altro processo) e quelle in consolidationIDs (processo corrente).

Il prompt di consolidazione

La sessione figlia riceve un prompt in inglese con regole rigide e risposta **JSON puro** (niente markdown):

```
{"new": [{"text": "...", "category": "user|project|workflow|preferences|decisions|status|enviro"}, {"update": [{"id": "...", "text": "..."}], "delete": ["..."], "summary": "..."}]
```

Regole: estrarre solo fatti reali (mai inventarli), ignorare il piccolo talk, scope project solo per fatti legati al codice in esame, summary di 2-5 frasi sull'utente. I tool di memoria sono **disabilitati** nella sessione figlia (purezza della risposta).

3.3 SURFACE — riemersione selettiva iniezione

L'hook experimental.chat.system.transform viene chiamato dal server a ogni richiesta LLM. Il plugin vi aggancia buildMemoryBlock — **selettivo per topic**:

1. Legge lo store; se vuoto, non fa nulla.
2. Estrae le **keywords del topic** dall'ultimo messaggio dell'utente (token ≥ 4 caratteri, stopword EN/IT escluse).
3. Le espande con expandTopicKeywords: sinonimi bilingue e varianti morfologiche (es. "linguaggi" → language, languages, linguaggio, linguaggi, code, codice, coding, programmazione, programming) —

colma il divario tra dedup semantico e retrieval lessicale senza embeddings.

4. Seleziona i fatti: core permanente — le prime 3 entry delle categorie operative (preferences, workflow, decisions, environment) sempre incluse; poi i fatti per `score()`, con **boost x3 per ogni keyword espansa** che compare nel testo del fatto.
5. Costruisce il blocco e lo appende a `output.system` (con cap `maxChars`).

```
<memory>
The following is long-term memory from previous conversations with this user...
<summary>L'utente preferisce Rust e lavora su plugin opencode...</summary>
<facts>
- [dreame] (preferences) The user's favorite color is green (verde).
- [explicit] (user) Il colore preferito dell'utente è il blu.
</facts>
</memory>
```

La sessione di consolidazione stessa è esclusa dall'iniezione (evita loop di memoria). Verifica end-to-end riuscita: "che ricordi hai di me?" → risposta con i fatti realmente ricordati; una sessione su topic estraneo (es. "una poesia sul mare") riceve solo il nucleo permanente e non viene contaminata.

4. Strutture dati

4.1 store.json — specifica unica

Campo	Tipo	Significato
id	uuid	Identificatore stabile della entry.
text	string	Il fatto, terza persona ("The user...").
category	enum	user · project · workflow · preferences · decisions · status · environment · other — unica lista valida. other è il fallback. general non è una categoria: le entry legacy con quel valore vengono migrate a other in lettura (normalizeStore).
scope	global project	Portata: globale o legata a un codice sorgente.
projectID	string?	Directory del progetto (per scope project).
weight	float	3 alla creazione esplicita (memory_write), 1 per i fatti sognati (consolidazione); +0.5 a ogni conferma; cap 4 per tutte le fonti.
lastSeen	epoch ms	Ultima conferma (usato dal decay).
created	epoch ms	Data di creazione.
source	explicit dreamed	Scritta dall'utente/tool o estratta dalla consolidazione. Gerarchia: explicit > dreamed.

Il summary è un campo top-level (max 3000 char) e deve contenere **solo il profilo operativo** dell'utente — mai fatti in conflitto con entry explicit. prune() elimina le entry con weight < 0.15 e taglia le entry oltre maxEntries.

4.2 state.json — commit a due fasi

```
{ "sessions": { "ses_xxx": 1786009535143 },  
  "InProgress": { "ses_yyy": { "targetTs": 1786009600000, "startedAt": 1786009605000, "chil
```

Due sezioni con ruoli distinti:

- sessions — sessionID → lastCompletedTs: la guardia (3.2) confronta max(info.time.completed, info.time.created) dei messaggi con questo valore; se la conversazione non è cambiata, salta. Il valore viene **committato solo a consolidazione riuscita** (dopo applyConsolidation + writeStore).

- `inProgress` — marcatori transitori della consolidazione in corso: `targetTs` (il `lastTs` obiettivo), `startedAt`, `childID` della `child` `headless`. Registrato **prima** di creare la `child`, rimosso nel `finally`.

Semantica "almeno una volta" senza duplicati: un crash dopo la registrazione di `inProgress` lascia un marcatore scaduto (`startedAt` oltre `inProgressTimeoutMs`): lo `sweep` lo ignora, il `consolidate` rileva la scadenza ("recovering stale `inProgress`"), riconsolida e pulisce. Un crash prima della fase 2 non ha avanzato `lastCompletedTs` → la sessione viene ritentata. Se invece il `finally` ha committato, la guardia salta — nessuna doppia elaborazione.

5. Configurazione

Variabile	Default	Effetto
<code>OPENCODE_MEMORY_OFF</code>	<code>off</code>	1 disabilita il plugin.
<code>OPENCODE_MEMORY_DIR</code>	<code>~/.local/share/opencode/memory</code>	Directory di store/state/SUMMARY.
<code>OPENCODE_MEMORY_DEBUG</code>	<code>off</code>	1 attiva i log di traccia (eventi, consolidate, transform).
<code>OPENCODE_MEMORY_DELAY_MS</code>	90000	Debounce della consolidazione dopo l' <code>idle</code> (3s nei test).
<code>sweepStartMs</code> / <code>sweepIntervalMs</code>	20000 / 60000	Prima scansione / intervallo dello sweep (sessioni orfane + GC child).
<code>sweepBatch</code>	3	Quante sessioni processare per ciclo.
<code>OPENCODE_MEMORY_GC_CHILD_AGE_MS</code>	600000	Età minima di una child orfana prima della rimozione automatica (insieme all'inattività).
<code>OPENCODE_MEMORY_INPROGRESS_TIMEOUT_MS</code>	600000	Scadenza del marcatore <code>inProgress</code> : oltre questa soglia la consolidazione è

		considerata morta e la sessione viene recuperata.
transcriptChars / maxFacts / maxChars	6000 / 12 / 4000	Cap del trascritto, numero di fatti iniettati, dimensione del blocco memory.

Comando utente: `~/ .config/opencode/command/memory.md` definisce `/memory` per interagire con la memoria dalla TUI.

6. Dettagli tecnici e lezioni apprese

Tema	Lezione
Formato API messaggi	I timestamp sono in <code>m.info.time.completed / created</code> — non <code>info.time.updated</code> (che non esiste). Il primo guard usava il campo sbagliato e la consolidazione saltava in silenzio: è stato il bug più costoso del progetto.
Parti dei messaggi	Le parts non hanno <code>time.end</code> : la risposta si riconosce dall'ultima parte <code>text</code> del messaggio <code>role:assistant</code> (polling ogni 1.5s, timeout 120s).
Timeout SDK	Il client SDK disabilita il timeout HTTP (<code>req.timeout = false</code>): una richiesta appesa non fallisce mai da sola → serve un timeout esplicito attorno alle chiamate critiche.
Cache dello store	Un cache in-memory di <code>getStore()</code> si desincronizza dal file (le scritture della consolidazione non la aggiornavano, e al load poteva catturare uno store vuoto) → lettura sempre fresca dal disco ; il costo I/O è trascurabile a questa scala.
Multi-turno LLM	Con due prompt successivi nella stessa child, <code>waitForReply</code> deve filtrare per <code>after</code> (timestamp): senza, il secondo turno "rilegge" la risposta del primo (il dedup verificava su dati già visti).
Crash mid-consolidation	Commit a due fasi: <code>InProgress</code> registrato prima della child (con <code>childID</code>), <code>lastCompletedTs</code> committato solo dopo <code>writeStore</code> . Crash → marcatore scaduto → recupero al riavvio; la child orfana è rimossa dal GC (3.2). Semantica "almeno una volta" senza doppie consolidazioni.
Lock	Mai annidare <code>withLock</code> (deadlock): <code>readStore</code> legge senza lock (sicuro grazie al <code>rename</code> atomico), <code>writeStore</code> viene chiamata solo dall'interno di una sezione critica.
Hook system	<code>experimental.chat.system.transform</code> riceve <code>{ sessionId, model }</code> e muta <code>output.system</code> (array di stringhe) — funziona sia per TUI che per <code>prompt_async</code> .

7. Limiti e note operative

- **Provider instabile:** con deepseek-v4-flash-free il modello a volte tace 60-90s o risponde 503 ("request queue full"); il timeout di consolidazione (120s) copre il caso ma può rallentare lo sweep.
- **Privacy:** storage sempre locale; l'elaborazione LLM usa il provider configurato (eventualmente remoto). Per dati sensibili: provider locale o `OPENCODE_MEMORY_OFF=1` (vedi §1).
- **Dedup tra lingue (risolto):** il secondo pass LLM (`dedupCheck`) confronta i candidati con le entries esistenti in modo semantico: "The user codes in Rust." vs "L'utente scrive codice in Rust." vengono riconosciuti come duplicati (verificato e2e). Resta il costo di una chiamata aggiuntiva (~60-90s con provider lenti).
- **Gerarchia fonti:** `explicit` vince sempre: la consolidazione non può duplicarla, aggiornarla o cancellarla; i fatti in conflitto sono **ignorati** (non entrano né come entry né nel summary). La sostituzione di un fatto esplicito richiede `memory_update` esplicito.
- **Retrieval lessicale:** il matching tematico resta testuale (sinonimi bilingue + forme flesse); "linguaggi di sistema" trova la memoria su Rust solo se i gruppi di sinonimi la coprono. Passo successivo naturale: embeddings (vedi §9).
- **Sessioni orfane (gestite):** il GC elimina le child `memory-consolidation` orfane (età > soglia E inattive; esenti quelle con `inProgress` valido); una sessione interrotta a metà consolidazione viene **recuperata** al riavvio (marcatore `inProgress` scaduto → riconsolidazione). Trade-off: un processo molto lento può essere sopravanzato dal recupero di un altro processo dopo il timeout; l'ultimo commit vince e il dedup rende benigna l'eventuale doppia estrazione.
- **Nessun DB:** JSON + lock file: adatto a scale piccole; oltre ~1000 entry serve ripensare lo storage.
- **Sweep e sessioni storiche:** al primo avvio lo sweep può mettere in coda decine di sessioni vecchie mai consolidate: nel test si usano state pre-marcati o directory dedicate.

8. Playbook di verifica

```
# 1. Avvio isolato (memoria di test)
OPENCODE_MEMORY_DIR=/tmp/mem OPENCODE_MEMORY_DELAY_MS=3000 OPENCODE_MEMORY_DEBUG=1 \
opencode serve --port 4457 --print-logs

# 2. Sessione utente + messaggio
curl -X POST localhost:4457/session -d '{"title":"t"}'
curl -X POST localhost:4457/session/<sid>/prompt_async \
  -d '{"parts":[{"type":"text","text":"il mio colore preferito è il verde"}]}'

# 3. Attesa consolidazione (idle → debounce → dream → cleanup)
cat /tmp/mem/store.json          # entries source:"dreamed" + summary
```

```
cat /tmp/mem/state.json          # sid → lastTs

# 4. Headless: la child NON appare tra le root
curl "localhost:4457/session?roots=true"      # nessuna "memory-consolidation"

# 5. Riemersione
curl -X POST localhost:4457/session/<sid2>/prompt_async -d '{"parts":[{"type":"text","text":
# → la risposta cita i fatti memorizzati

# 6. Test automatico del conflitto (explicit + contraddizione)
# seed: explicit "The user's favorite color is green."
# conversazione: "my favorite color is green and I code in Rust"
# attesa → assert: nessuna nuova entry, summary privo del conflitto,
#           il duplicato dreamed eventuale viene eliminato
```

9. Evoluzione: verso una pull request a opencode

Il progetto è maturo per essere proposto a monte. Punti di leva e di attenzione:

Area	Stato oggi	Proposta PR
Sessioni figlie	parentID accettato dal server, usato "sotto banco"	Documentare <code>SessionCreateData.parentID</code> ; valutare un flag hidden esplicito nelle sessioni (esclude anche il picker con <code>roots=true</code> e l'elenco "recenti").
Hook system	<code>experimental.chat.system.transform</code> già esistente e stabile	Promuoverlo (rimuovere il prefisso <code>experimental</code>) con test dedicati; documentare <code>input.sessionID</code> .
Eventi	<code>session.idle/session.status</code> presenti	Aggiungere un evento esplicito <code>"session.ended/idle"</code> nel tipo <code>Session.Event</code> per evitare l'uso di due eventi ridondanti.
Plugin pack	Script singolo in config utente	Estrarre <code>opencode-memory</code> come pacchetto npm installabile (<code>opencode plugin add opencode-memory</code>), con README e test e2e automatici (il playbook di cui sopra).
Dedup	Secondo pass LLM nella stessa child headless (funzionante, costo ~60-90s)	Usare embeddings (chiamata <code>provider.small_model</code> già esposta) per il dedup cross-lingua senza costo LLM; in alternativa confronto vettoriale diretto nel plugin.
Schema dati	Uniformato (categorie, pesi, other come fallback, migrazione <code>general→other</code>)	Validazione JSON schema (Zod) condivisa tra plugin e test; test automatici sui conflitti explicit/dreamed.
Privacy	Storage locale, inference via provider configurato	Documentare esplicitamente nel README la distinzione; opzionale: modalità local-only (blocco di DREAM con provider remoti).

Raccomandazione: iniziare dalla PR "docs + test": documentare parentID e l'hook system, con un test che crea una child e verifica l'invisibilità in `roots=true`. È il passo a più basso rischio e a più alto valore per la community. La checklist per proporre l'intera memoria al core: schema validato con test, GC delle child nel core, policy privacy esplicita.